Phase 3 Report

Osama Najeeb, Xinyang Zhang

Applied Programming and Computational Thinking – CSSE 5210

Avishek Mukherjee

University of Detroit Mercy

02/26/2026

Abstract

Potholes are a major issue in Detroit, frequently causing severe damage to vehicle tires and alignments. Since manual road inspections are slow and inefficient, our project aims to automate pothole detection using a portable camera-based system. We trained lightweight YOLO-based object detection models on a GPU-equipped Windows PC using a combination of public pothole datasets and our own road footage collected around the university campus. To identify a strong starting point for deployment, we compared multiple training configurations and model variants (e.g., different YOLO model sizes and

architectures) using the same validation split. Based on these experiments, we selected the YOLO26n as our preferred pretrained model because it provided the best overall detection performance among the tested options.

For Phase 3, we also focused on validating a real-time end-to-end prototype. A Raspberry Pi 5 equipped with a camera module captures live video frames and transmits selected frames over Wi-Fi to a PC-hosted Flask inference service. The PC performs YOLO inference and returns detections as structured JSON (bounding box coordinates and confidence values). The Raspberry Pi receives the JSON response, prints it for verification, and renders bounding boxes on the live stream in real time (also viewable via touchscreen and RealVNC for recording). The main objective for the next phase is to reduce end-to-end latency, improve far-range detection robustness, and explore options for running inference fully on the Raspberry Pi for continuous real-time monitoring.

Sensor Integration Proof of Concept

Hardware Setup:

Currently our setup is a hybrid architecture that uses a PC (NVIDIA GeForce RTX 5070 Ti), mobile camera, and Raspberry Pi 5 equipped with a Raspberry Pi camera module 3. The Raspberry Pi is powered via USB-C (5V adapter), using a laptop USB charger as the power source. This setup demonstrates that the system can be operated portably and remotely without relying on a fixed power outlet, which supports continued testing beyond Phase 3. The Raspberry Pi connects to the local network via Wi-Fi.

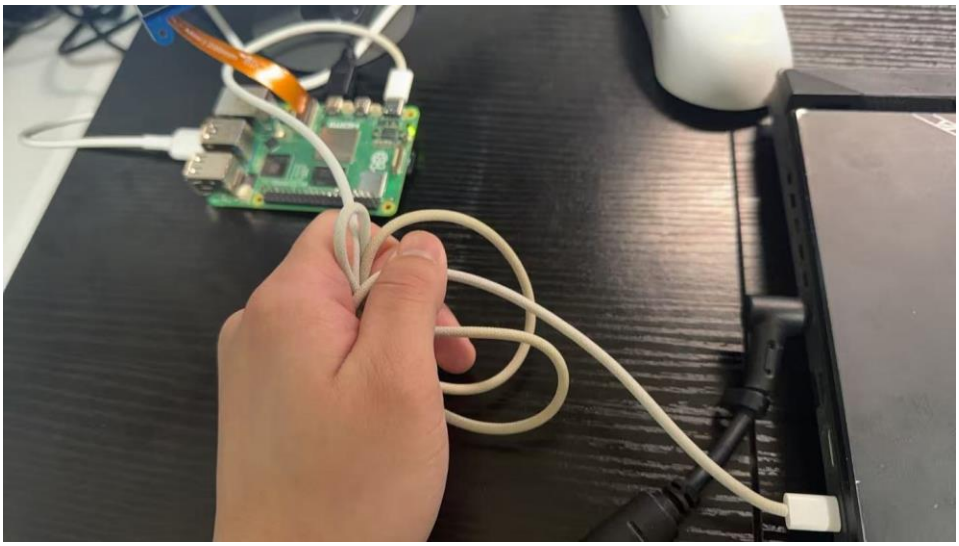
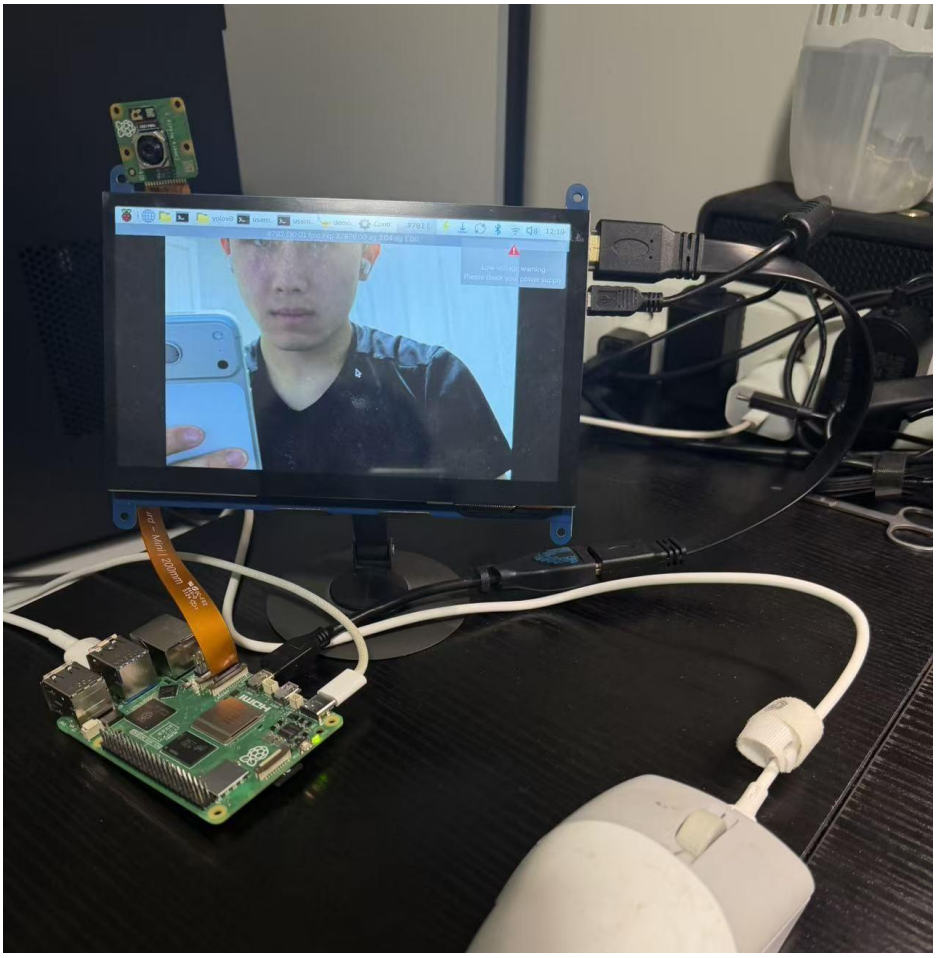
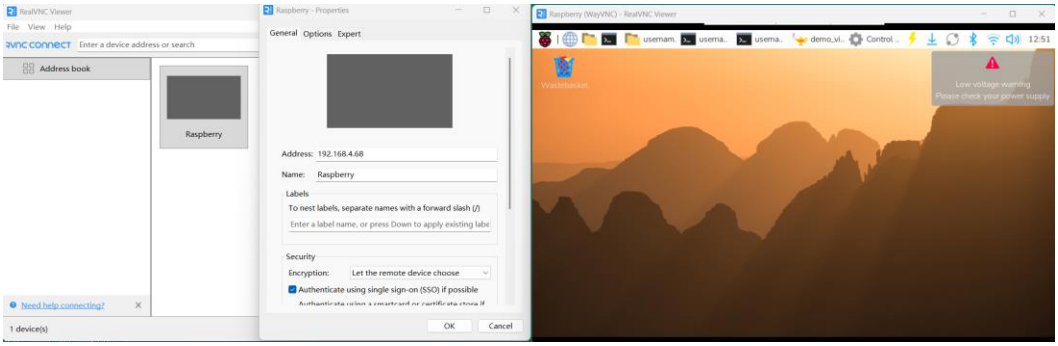


Fig. 1. Raspberry Pi powered via USB-C (5V adapter) using a laptop USB charger.

In addition, the Raspberry Pi is connected to a touchscreen display, enabling real-time local monitoring of the camera feed and detection outputs during experiments. A Windows PC serves as the client device to record and view the streamed video and to visualize the detection results.



Furthermore, the Windows PC mirrors the Raspberry Pi desktop using RealVNC Viewer. This configuration makes it easy to record demonstrations, capture screenshots for documentation, and monitor the on-device interface (camera preview and detection outputs) while the system is running.



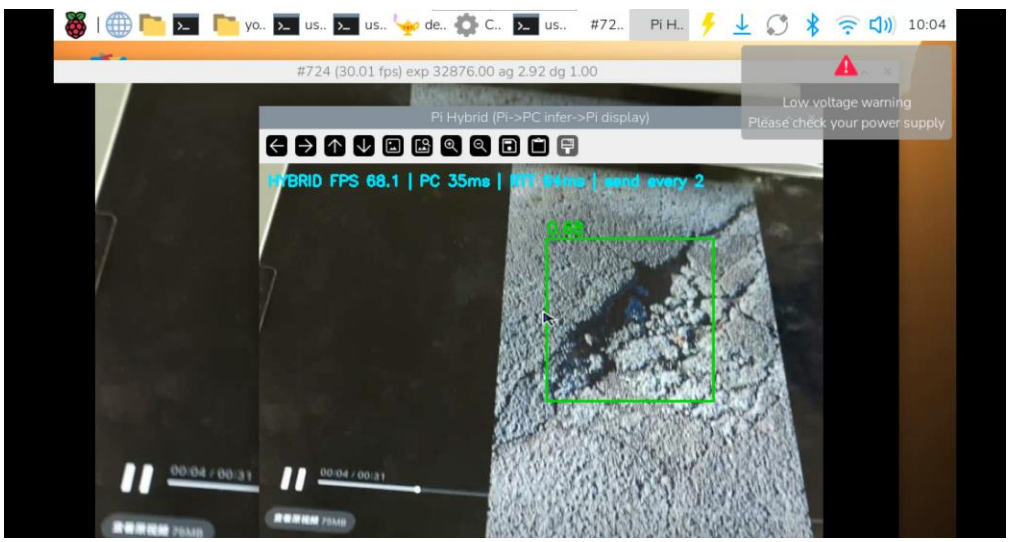
Our PC was used to perform computationally intensive tasks of training and validating the YOLO deep learning model, PCs dedicated GPU and memory were used to handle massive processing workloads to build a model.

Mobile Camera was used to capture unseen potholes videos within campus parking lot and outside campus to test our model.

Raspberry Pi 5 & Camera Module are our target edge-computing device. For testing, we set up a hybrid communication where the Pi and PC talk to each other. We successfully conducted tests by directing the connected Pi camera module at the tablet screen which showed the recorded pothole videos that we did with our mobile camera. The system accurately detects the potholes and draws lime green bounding boxes around them in real-time directly from the screen.

Captured Data Type: The sensor captures continuous RGB video frames (images) for downstream processing and object detection.

Evidence:



As you can see from the screenshot, we were using a camera module and raspberry pi to detect potholes on our tablet screen. Also, you can check out the video of our demo demo.mp4

Data Pipeline & Communication

Computation Location:

Hybrid setup (Client-Server Architecture). The Raspberry Pi captures the footage and sends it to the PC while PC does most of the heavy lifting by detecting potholes with the help of a trained model since it has high memory and GPU.

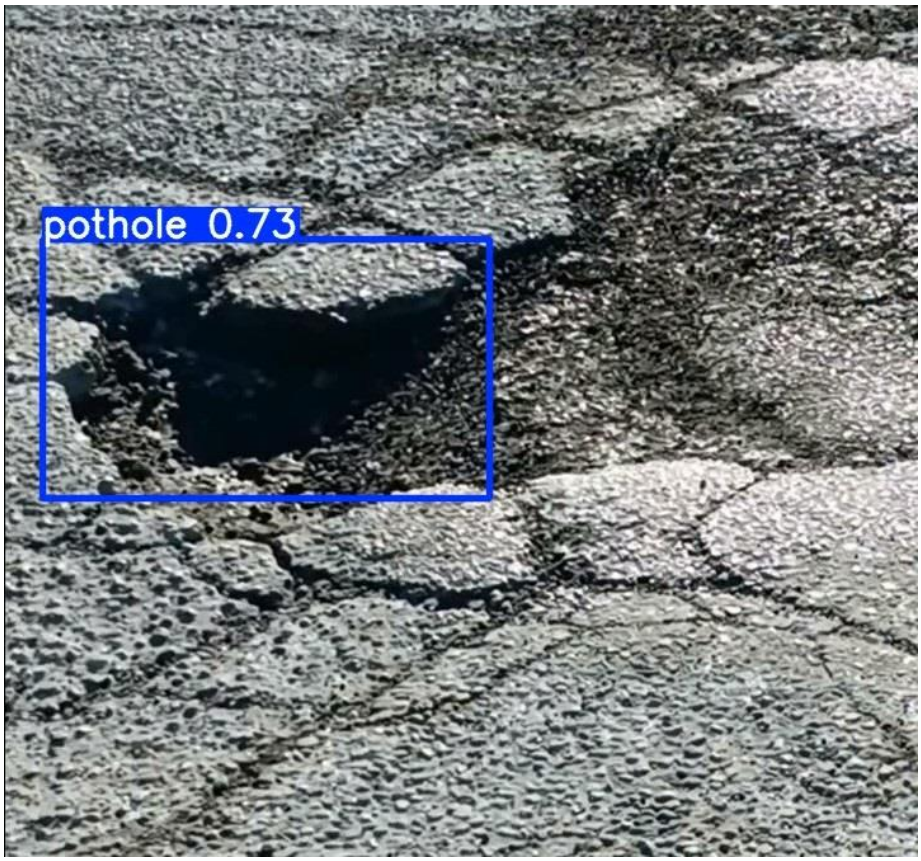
Data Pipeline:

Our system uses a hybrid real-time pipeline to transfer camera frames from a Raspberry Pi 5 to a Windows PC for inference and then return detection results back to the Raspberry Pi for visualization. The Raspberry Pi continuously captures RGB frames from the camera, encodes selected frames as JPEG images, and sends them to the PC over Wi-Fi using an HTTP POST request. The Windows PC hosts a Flask inference service that receives the uploaded image, performs YOLO-based pothole detection, and returns the results as a structured JSON response. The Raspberry Pi parses the returned JSON (bounding box coordinates and confidence values), prints the received JSON in the terminal for verification, and overlays bounding boxes on the live video stream in real time.

End-to-end flow:
Raspberry Pi camera → JPEG encoding → HTTP POST (Wi-Fi) → PC Flask /infer → YOLO inference → JSON response → Raspberry Pi draws boxes & displays annotated frames

Basic Visualization of Incoming Data

The project uses two forms of data visualization to monitor both live incoming sensor feeds and machine learning metrics. The primary visualization of our incoming sensor data is handled via an OpenCV video display window. As the Raspberry Pi streams live camera frames to the PC, OpenCV renders the visual data in real time, overlaying the inference results such as bounding box and confidence percentages directly onto the detected potholes.



Initial Pre-Processing of sensor data

Baseline Computer Vision Operation (Frame Decoding & Resizing): Since we're using a hybrid client-server setup, the raw bytes coming in over the Wi-Fi socket from the Raspberry Pi can't go straight into the model. The PC first uses OpenCV to decode those bytes back into a usable image. From there, each frame gets resized to 640×640 pixels to match what the YOLO network expects as input.

Threshold/Event Detection (Confidence Filtering): To avoid false detections, we added a confidence threshold to the inference script. The model only flags something as a pothole and draws a bounding box around it if it's at least 50% confident. Anything below that gets ignored, which helps cut down on noise from road surfaces that might otherwise trigger a false positive.

Machine Learning / Data Science Plan

ML/Deep Learning Method

This project uses deep learning for real-time object detection. We implemented the YOLO (You Only Look Once) Nano architecture. Rather than simple classification or regression, object detection is necessary here because the system needs to both identify and localize potholes within a video frame in real time drawing bounding boxes around detected anomalies as they appear in the live road feed.

Supervised OR Unsupervised

For now, we are focusing on supervised learning methods as we have labelled images where potholes are already annotated with the bounding boxes, allowing the network to learn the visual features that distinguish a pothole from normal road surface.

Dataset

Training data is sourced from an existing annotated dataset obtained via Roboflow, which contains labelled pothole images across varied road conditions and lightning environments. For real world testing and validation, we used mobile cameras to record potholes outside the campus and parking lot of campus.

Evaluation

We evaluate the model using several metrics. mAP@50 and mAP@50–95 measure detection accuracy based on bounding-box overlap with ground truth, while Precision and Recall track false positives versus missed detections. We also evaluate real-time performance using inference time, end-to-end latency, and FPS. These runtime metrics are measured in both our hybrid pipeline (Pi → PC → Pi) and exploratory on-device Raspberry Pi 5 tests. Inference time reflects model compute only, while end-to-end latency also includes capture, encoding, network transfer, and visualization.

Model	Training Config	Params	GFLOPs	P	R	mAP50	mAP50-95	Infer (ms)
YOLOv8n	ep=30, batch=8	3,005,843	8.1	0.83	0.71	0.79	0.428	0.5
YOLOv8n	ep=80, batch=16, AMP	3,005,843	8.1	0.89	0.74	0.82	0.502	0.6
YOLOv8n	ep=150, patience=30	3,005,843	8.1	0.89	0.73	0.81	0.513	0.5

YOLOv8s	ep=80, batch=16, AMP	11,125,971	28.4	0.90	0.71	0.82	0.523	1.2
YOLO26n	**ep=80, batch=16, AMP**	**2,375,031**	**5.2**	**0.87**	**0.75**	**0.84**	**0.543**	**0.8**
YOLO26m	ep=80, batch=16, AMP	20,350,223	67.8	0.89	0.72	0.83	0.510	2.9

Research Questions

This project explores the feasibility of automating road inspections by deploying a standalone object detection system on edge hardware. Specifically, the technical question such as How can we effectively deploy a lightweight YOLO Nano object detection model directly onto a resource-constrained device (a Raspberry Pi 5) to identify potholes without relying on a powerful PC or external network for heavy computational lifting?

The AI/ML Challenge

- **Real-Time Constraints (Latency):** The core hurdle is getting the model to process video feeds at a viable frame rate (FPS) on a device that lacks a dedicated graphics card. The standalone system currently suffers slow processing speeds compared to a hybrid setup that streams to a PC.
- **Model Optimization and the Accuracy/Efficiency Trade-off:** The project actively addresses the challenge of optimizing an AI model to run faster on limited hardware while finding the right balance between maintaining high detection accuracy and achieving smooth video performance.
- **Generalization to Real-World Environments:** By combining public datasets with custom video footage recorded locally, the project also tackles the challenge of ensuring the model generalizes well enough to reliably detect specific, real-world road hazards like Detroit potholes.

Timeline

What is complete now

- Camera integration on Raspberry Pi 5 is working; live frames can be captured reliably (local monitoring via touchscreen).
- Hybrid end-to-end pipeline is functional: **Pi captures frames → sends frames over Wi-Fi to PC (Flask) → PC runs YOLO inference → returns YOLO-style JSON → Pi receives JSON and draws bounding boxes in real time**.
- RealVNC Viewer is set up to mirror the Raspberry Pi desktop on the Windows PC for recording and screenshots.
- Model experiments were conducted and compared across multiple configurations and model families (e.g., YOLOv8 and YOLO26), and **YOLO26n** was selected as the preferred model based on validation performance.
- On-device testing was attempted: running inference fully on Raspberry Pi 5 was tested, but real-time performance was not smooth enough for the final demo, so the **hybrid approach was chosen** as the primary deployment strategy.

Timeline (2/16 – 3/2+)

Feb 16 – Feb 22 (Model comparison + hardware debugging)

- Train and validate multiple models/configurations (YOLOv8 variants vs YOLO26 variants).
- Summarize results (precision/recall/mAP + qualitative video testing) and select final model (**YOLO26n**).
- Hardware debugging and deployment testing:
    - Verify camera capture stability and display setup (touchscreen + RealVNC).
    - Test feasibility of **fully on-device inference on Raspberry Pi 5** (no PC).
    - Conclusion: on-device inference was functional but not real-time enough; finalize **hybrid mode (Pi→PC→Pi)** for stable performance.

Feb 23 – Mar 1

- Prepare a stable, repeatable demo workflow:
    - Run the hybrid pipeline smoothly (consistent streaming, stable Wi-Fi, clear visualization).

    - Record a clean 2–5 minute demo video with evidence (Pi terminal JSON output + PC Flask logs + annotated video).
- Finalize the “Phase 3 report” sections:
    - Evidence screenshots, model comparison summary, and pipeline explanation.
    - Confirm GitHub repo is updated and ready for submission.

After Mar 2

- Improve far-range pothole detection (small-object challenge) using:
    - higher input resolution, ROI cropping/tiling, and targeted fine-tuning with in-domain data.
- Optimize hybrid system performance:
    - reduce end-to-end latency, improve FPS stability, and reduce dropped frames.
- Continue exploring edge deployment options:
    - acceleration methods (e.g., quantization/compression and optimized inference backends) to reassess whether real-time on-device inference is feasible.
- Prepare semester-end deliverables (poster/presentation content):
    - refine project story, results, and final demonstration assets.

What will be finished before the final demo

- Final model confirmed (YOLO26n weights locked).
- Hybrid pipeline demo runs smoothly with stable FPS/latency.
- Evidence captured: Pi JSON logs + PC Flask request logs + annotated frames screenshots.
- Demo video recorded (2–5 min) with clear narration and system proof.
- GitHub repo updated (code + instructions + results summary).

Challenges & Risks

During Phase 3, our main challenges were related to dataset consistency, small-object detection in real videos, and real-time performance on edge hardware. First, we attempted

to merge four pothole datasets into a single training set (~11k images with YOLO labels), but model performance was worse than expected. After inspection, we found significant annotation inconsistency across datasets (some with very large/loose bounding boxes and others with tight boxes), which likely hurt convergence and generalization. In addition, several datasets contained many augmented near-duplicates, and merging them introduced redundancy and noise. As a result, we switched to a single dataset with a more consistent labeling style and used its official train/val/test split (https://universe.roboflow.com/yeeun-kim-fyvoj/pothole-vhmow/dataset/17).

Second, qualitative video testing showed strong results for close-range potholes but weaker performance for far-range potholes. This is mainly a small-object challenge (distant potholes occupy only a few pixels), and resizing to a fixed inference size (e.g., 640) can further reduce distant detail. There is also a domain gap between curated dataset images and real-world road videos under varying lighting, motion blur, and camera angles.

Finally, we tested running inference fully on the Raspberry Pi 5 (on-device deployment). While the model can run on the Pi, the real-time performance was not as smooth as the hybrid setup, resulting in lower FPS and higher latency. This is a risk for the final real-time demo if we require full on-device inference. Therefore, our current approach prioritizes the hybrid Pi→PC pipeline for real-time responsiveness, while we continue exploring edge acceleration options for Phase 4 (e.g., higher-efficiency models, quantization, and optimized inference backends).

AI use

We used generative AI tools (ChatGPT and Claude) as a support resource during Phase 3. Specifically, we used AI to (1) brainstorm and refine our project plan and system architecture (Raspberry Pi → PC inference → Raspberry Pi visualization), (2) improve technical writing quality in the report (clarity, organization, and grammar), and (3) help draft and polish short sections such as the abstract, timeline, and challenges/risks. AI was also used to suggest debugging approaches and implementation ideas (e.g., where to add logging for JSON responses and how to present evidence for communication and visualization).

All final design choices, model training runs, parameter selection, data preparation, experiments, and performance measurements were completed and verified by our team.

We reviewed, tested, and modified all AI-suggested content before incorporating it into the final submission.